

Requirements

- JRE system Library
- JUnit 4

Part 1 Scanner.getNextToken(), Scanner.reserved, TokenInfo, ScannerTests

- I removed the default packages in Jgrasp, as it was giving bugs.
- Gave tests/spim a different package name
- Wrote Junit tests to see if my code works in ScannerTests
- I copied Scanner to ScannerBackup just in case I need it.
- Added multiline support.
- Updated reserved hash table with all reserved keywords
- Updated tokenInfo with lexemes for the literals keywords and operators
- Add DIV and DIVASSIGN to comment check section.
- Added the reaming operators, and if then for operators, that have more than one char
- Added float long and double terminals, and made sure it can handle floats that don't start with an int
- Added support for hex and binary, but not yet for octals, because of how it's parsed with a just a 0, and added reporting for malformed literals.

Part 2

- Created JUnit tests for parser
- Added parsing support for double and float literals in primary() and literal()
- Created JDoubleLiteral and JFloatLiteral AST node classes
- Added parsing support for long literals in primary()
- Created JLongLiteral AST node class
- Implemented parsing for all compound assignment operators in assignmentExpression()
- Added AST node classes for: JPlusAssignOp, JMinusAssignOp, JStarAssignOp, JDivAssignOp, JRemainderAssignOp, JBitAndAssignOp, JBitOrAssignOp, JBitXorAssignOp, JLeftShiftAssignOp, JRightShiftAssignOp, JUnsignedRightShiftAssignOp
- Added parsing for conditional expressions in conditionalExpression() using ternary ? :
- Created JConditionalExpression AST node
- Implemented forStatement() with lookahead to distinguish basic vs. enhanced for-loops
- Created JForStatement and JEnhancedForStatement AST nodes
- Added helper seeLocalVariableDeclaration() to distinguish declarations in for-init
- Fixed scanner position restoration in lookahead to prevent misclassification of JVariable
- Implemented switchStatement() with support for case and default labels
- Created JSwitchStatement and SwitchStatementGroup classes
- Implemented tryStatement() with support for multiple catch blocks and optional finally
- Created JTryStatement AST node
- Implemented throwStatement() for throw expressions
- Created JThrowStatement AST node

- Added `parseThrowsClause()` to support throws clauses in method declarations
- Extended `JMethodDeclaration` to accept and store `throwsList`
- Verified all features with debug prints and trace diagnostics in `statement()` and `statementExpression()`
- Ensured all new constructs produce valid AST nodes and integrate with existing parser structure
- Added static boolean to help with expressions inside for loop declarations
- Added debug statements

Part 3

- Made JUnit tests called `JVariableTest.java`
- Improved JUnit test and moved old one to `JVariableTestBackUp.java`
- I used `java.lang.reflect.Field` to get private fields
- Backed up `JVariable` and `JCompilationUnit`
- Added getter for mods in `JClass` declaration
- Added new fields to `JCompilationUnit`
- Added checking for only one public type
- Improved error reporting
- Updated error reporting and cleaned up files

Part 4

- Added `Part4IntegrationTest.java` (code generation) which actually creates the java and class files, runs them and compares output and checks the bonus exercises, it also puts files in the user's temp directory as to not clog the workspace
- Added `Part4Test.java` to do some light tests (no code generation) to make sure it is scanning and parsing correctly
- Added do while in `JDoStatement.java`
- Added `||`
- Added `JTypeExpression` which acts as a wrapper for `TypeName` and a child of `JExpression` allowing static methods and fields in primary expressions
- 'for' and conditional expressions were given codegen
- Did codegen methods for '<', '>=' and '++'
- For Debugging I used `javap` on the generated files in the Windows Powershell to see the bytecode and what needed to be added
- Started Bonus Exercises
- Added extra credit check to `Part4IntegrationTest.java`
- Implemented more codegen methods
- Redid `JBinaryExpression` children to handle longs